

Programação de Sistemas

Revisão da Linguagem C Padrão - Módulo 4 -

Prof. Ronaldo Augusto de Lara Gonçalves

Agradecimentos ao Prof. Marcelo de G. Malheiros

Variável Vetorial (Array) ou Vetor



- São variáveis compostas homogêneas, que podem armazenar uma quantidade de elementos de mesmo tipo, sendo que os elementos residem em posições diferentes do vetor e cada um pode ser acessado pelo número da posição (**índice**). Portanto, são variáveis indexadas.
- Um vetor é criado/declarado utilizando um nome e um tamanho (quantidade de elementos) entre **colchetes**.
- Uma inicialização (atribuição de valores) pode ser feita já na declaração, caso contrário, a inserção de elementos deverá ser feita durante a execução, assim como a remoção.
- Cada
- A numeração dos índices sempre começa em 0 e finaliza em $n-1$, sendo n o número total de elementos.
- O espaço total de memória de um vetor equivale a somatória de todos os espaços individuais alocados subsequentemente.

Vetor



» Representação Gráfica, declaração e operações:

v	7	1	5	4	0	8	12	-3	10	2
	0	1	2	3	4	5	6	7	8	9

```
int v[10] = {7, 1, 5, 4, 0, 8, 12, -3, 10, 2};
```

```
v[8] = v[5] + v[0];  
scanf("%d",&v[3]);  
for (i=0; i<10; i++)  
    printf("%d, ",v[i]);
```

- O tamanho pode ser especificado por uma variável, que pode ser fornecida pelo usuário.
- **Atenção:** isso vale apenas para compiladores **mais recentes** da linguagem C, que tenham o recurso VLA (**Variable Length Arrays**). Entretanto, esses vetores não podem ser inicializados na declaração. Ex:
 - `int tamVet;`
 - `printf("Especifique o numero de notas: ");`
 - `scanf("%d", &tamVet);`
 - `float notas[tamVet];`
 - `for (i = 0; i < tamVet; i++) {`
 - `notas[i] = 0.0;`
 - `}`
- O vetor funciona como um *buffer*, ou seja, um banco de reserva de espaço que pode ser todo utilizado ou não. A forma de uso dos espaços do vetor depende dos objetivos do programa.

Exercícios/Desafios – Parte 01



- Faça o seguinte programa: declare um vetor de 10 posições inteiras e inicialize todas elas com 0 (zero). Depois, atribua um valor diferente para cada posição. Calcule e mostre a soma dos elementos de índice par e faça o mesmo com os elementos de índice ímpar. Não use comandos de repetição. Entregue o código fonte no AVA.
- Faça o seguinte programa: peça *n* para o usuário, então, declare um vetor Vet de *n* números reais. Use repetição com *for* para preencher o vetor com valores fornecidos pelo usuário. Somente depois de preenchido, calcule a media aritmética entre os elementos do vetor usando *while*. Somente depois de calcular a média, localize o maior e o menor elementos do vetor, usando *do-while*. **Atenção:** impeça o usuário de digitar números repetidos. Entregue o código fonte no AVA.

Exercícios/Desafios – Parte 02



- Faça um programa para calcular o produto escalar. O produto escalar é obtido com a multiplicação de dois vetores de mesmo tamanho “lógico”, elemento a elemento, e somando tudo, resultando em um único valor inteiro. Declare 2 vetores com tamanho máximo. Peça para o usuário o tamanho “lógico” n dos vetores que serão armazenados, respeitando o tamanho máximo da alocação “física”. Gere os elementos aleatoriamente e mostre-os na tela. Mostre o produto escalar. Faça um loop com pergunta para repetir o programa novamente. **OBS:** use os comandos `srand()` e `rand()` da biblioteca `<stdlib.h>` e `time(NULL)` da biblioteca `<time.h>`. Investigue os comandos e bibliotecas. Entregue o código fonte no AVA.
- Faça um programa que verifique se uma sequência de caracteres fornecida pelo usuário é palíndromo. Use apenas um vetor de tamanho máximo fixo para armazenar a sequência. O tamanho da sequência e a própria sequência devem ser fornecidos pelo usuário. Faça um loop com pergunta para repetir o programa novamente. Entregue o código fonte no AVA.

Exercícios/Desafios – Parte 03



- Faça o seguinte programa: verifique se dois vetores v1 e v2, de inteiros, de mesmo tamanho, possuem exatamente os mesmos elementos, independente da ordem entre eles. Considere a possibilidade de haver elementos repetidos. O tamanho lógico e os elementos devem ser digitados pelo usuário. Defina um tamanho máximo para a alocação física. Faça um loop com pergunta para repetir o programa novamente. Entregue o código fonte no AVA.
- Faça o seguinte programa: gere aleatoriamente um vetor de n inteiros, sendo n fornecido pelo usuário. Mostre-o no vídeo. Depois, ordene o vetor usando Seleção Direta. Mostre-o no vídeo. Trabalhe com tamanhos lógico e físico. Faça um loop com pergunta para repetir o programa novamente. Entregue o código fonte no AVA.
- Idem para Bubble Sort.

Exercícios/Desafios – Parte 04



- Faça o seguinte programa: localize um vetor v1 dentro de um vetor v2, tal como localizar uma palavra dentro de uma frase. Trabalhe com tamanhos lógico e físico. Faça um loop com pergunta para repetir o programa novamente. Defina um máximo para a alocação. Entregue o código fonte no AVA.
- Faça o seguinte programa: faça um menu com as opções: 1) Insere Elemento, 2) Remove Elemento, 3) Ordena Elementos, 4) Mostra Elementos e 5) Finaliza Execução. Trabalhe com tamanhos lógico e físico. Faça um loop com pergunta para repetir o programa novamente. Entregue o código fonte no AVA. **Obs:** a inserção é feita após o último elemento e aumenta o tamanho lógico. A remoção arrasta os elementos posteriores e diminui o tamanho lógico. O tamanho físico é imutável.

- Execute:
 - Experimento_01
 - Experimento_02

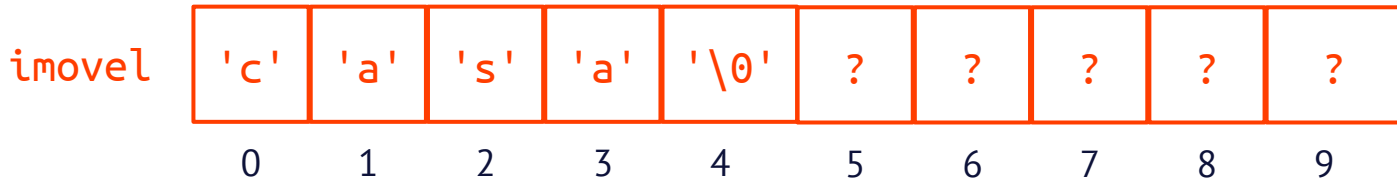
Variável String (vetor de caracteres)



- Uma variável do tipo *string* é armazenada como um vetor de caracteres, declarada da mesma forma que qualquer variável vetorial, tal como o seguinte exemplo:

```
char imóvel[10];
```

- Ela pode ser manipulada da mesma forma que qualquer vetor, mas para ser manipulada com funções específicas para *string*, ela deve armazenar uma sequência de caracteres finalizada com o caractere terminador `'\0'` (barra invertida zero), conforme a figura a seguir, a qual mostra o armazenamento da *string* "casa".



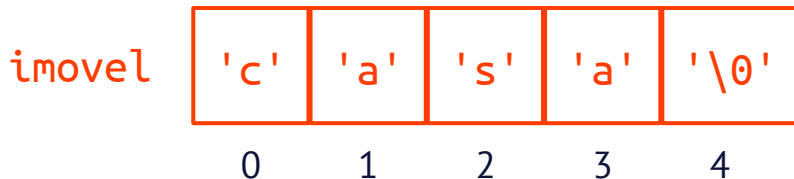
Variável String (vetor de caracteres)



- O terminador informa onde é o final da *string* para as funções de manipulação de *strings* como *printf*, *scanf*, *strcpy*, *fgets*, *strcmp*, *strcat* e *strlen*. O tamanho da *string* na declaração deve prever espaço para o terminador.
- Assim como qualquer vetor, uma *string* não precisa ocupar todo o espaço alocado na declaração.
- Podemos omitir o tamanho da *string* na declaração com inicialização, tal como segue.

```
char imovel[] = {'c', 'a', 's', 'a', '\\0'};
```

- Nesse caso, a *string* é criada com o tamanho definido pelo numero de caracteres da inicialização, incluindo o terminador. Nesse exemplo, 5, indexados de 0 a 4.



Variável String (vetor de caracteres)



- Uma *string* também pode ser inicializada na declaração usando uma sequência de caracteres entre aspas duplas. Nesse caso, a inserção do terminador '`\0`' é feita automaticamente, de forma implícita, tal como segue:

```
char imovel[] = "casa";
```
- Assim como vetor, o identificador/nome de uma string “contém” o endereço inicial da alocação na memória (endereço do primeiro elemento, de índice 0).

Operações com Strings



» Leitura com `scanf()`

- Ex: `scanf("%s", imovel);`
- Informar o formato "%s" e o identificador da variável *string* sem &. A *string* é lida toda de uma vez, na medida em que as teclas são digitadas, até a digitação da tecla <enter>. No lugar da tecla <enter>, a função *scanf* coloca um terminador.
- Cada caractere digitado ocupará uma posição da variável *string* conforme a sequência de digitação, a partir do índice 0.
- *scanf* não aceita a leitura de *strings* com espaço em branco, sendo que somente a sequência inicial antes do espaço em branco será aproveitada, ficando o restante no *buffer* do teclado.
- *scanf* ignora os espaços em branco que forem digitados no início
- Permite limitar os caracteres lidos, por ex: `scanf("%5s", imovel);`

- Faz a leitura de tudo que foi digitado, inclusive do `<enter>` final.
- Precisa especificar a quantidade máxima a ser lida.
- Não precisa especificar o tipo do dado a ser lido, pois ela é específica para a leitura de *strings*.
- É utilizada para a leitura de arquivos, mas pode ser utilizada na leitura em teclado informando a entrada padrão (*stdin*).
- Pertence a biblioteca padrão de entrada/saída: *stdio.h*
- Ex:
 - `char imovel[10];`
 - `fgets(imovel, 10, stdin);`

- A impressão é feita de uma única vez, elemento a elemento, a partir do índice 0 até o índice anterior ao caractere terminador, conforme segue:
 - `printf("%s", imovel);`
- Uma *string* também pode ser impressa com um laço, como em qualquer vetor, conforme segue. Nesse caso, o programador controla quando parar a impressão.
 - `for (i = 0; imovel[i] != '\0'; i++) {`
 - `printf("%c", imovel[i]);`
 - `}`
- Uma *string* sem caractere terminador poderá causar um erro de *segmentation fault* ao ser manipulada pelas funções de manipulação de strings *printf*, *strcpy*, *strlen* e outras.

- Função disponível na biblioteca `<strings.h>`
- Copia a *string* `s1` para a *string* `s2`, ou melhor, copia a *string* armazenada no endereço de `s1` para o endereço armazenado em `s2`
- A cópia é feita até o caractere terminador de `s1`
- A variável `s2` precisa ter espaço suficiente para receber a *string* `s1`
- Ex:
 - `char s1[15]="teste", s2[15];`
 - `strcpy(s2, s1);`
 - `printf("%s",s2);`
 - `strcpy(s1, "novo teste");`
- Investigue outras funções: `strlen()`, `strcmp()` e `strcat`

- Alocação e dealocação feita durante a execução, usando *malloc()* e *free()*
- Usa-se de forma similar a alocação estática, com leve ajuste no formato da declaração e do acesso
- Declara-se uma variável ponteiro para o tipo a ser armazenado, usando o símbolo * (asterisco) a esquerda do nome da variável
- Uma variável do tipo ponteiro armazena o endereço do local onde estará o dado e não o dado propriamente dito
- Exemplo:

```
int *v; // aqui dizemos que v é uma variável ponteiro ou apenas ponteiro
v = malloc(10*sizeof(int)); // aloca 10 posições para conter inteiros
v[5]=15;
printf(("v[5] = %d", v[5]));
free(v); // devolve o espaço recebido e não pode mais utiliza-lo
```

Perguntas?